

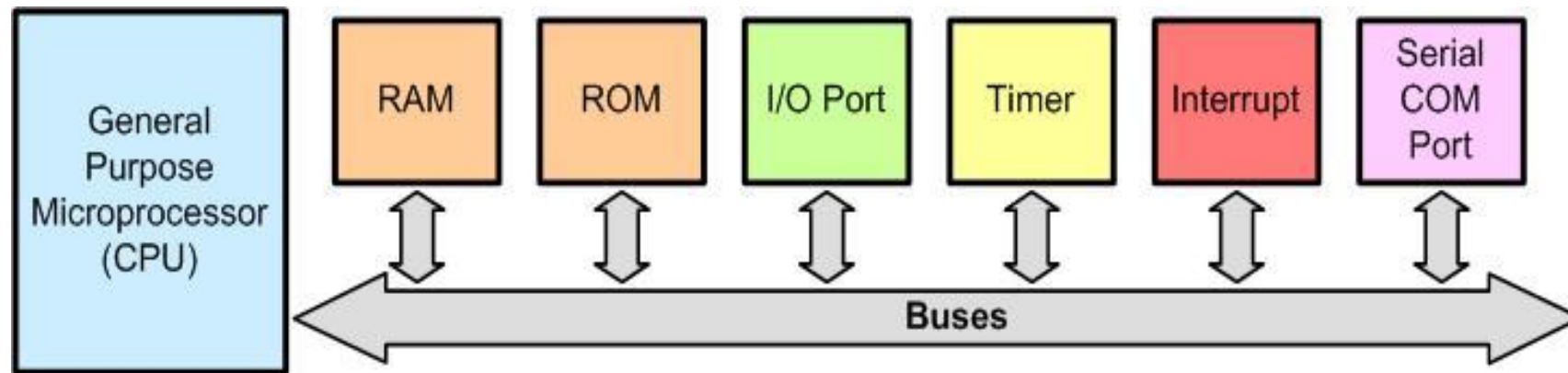
Module 2

ARM Microcontroller RISC Architecture, Registers, Memory Map

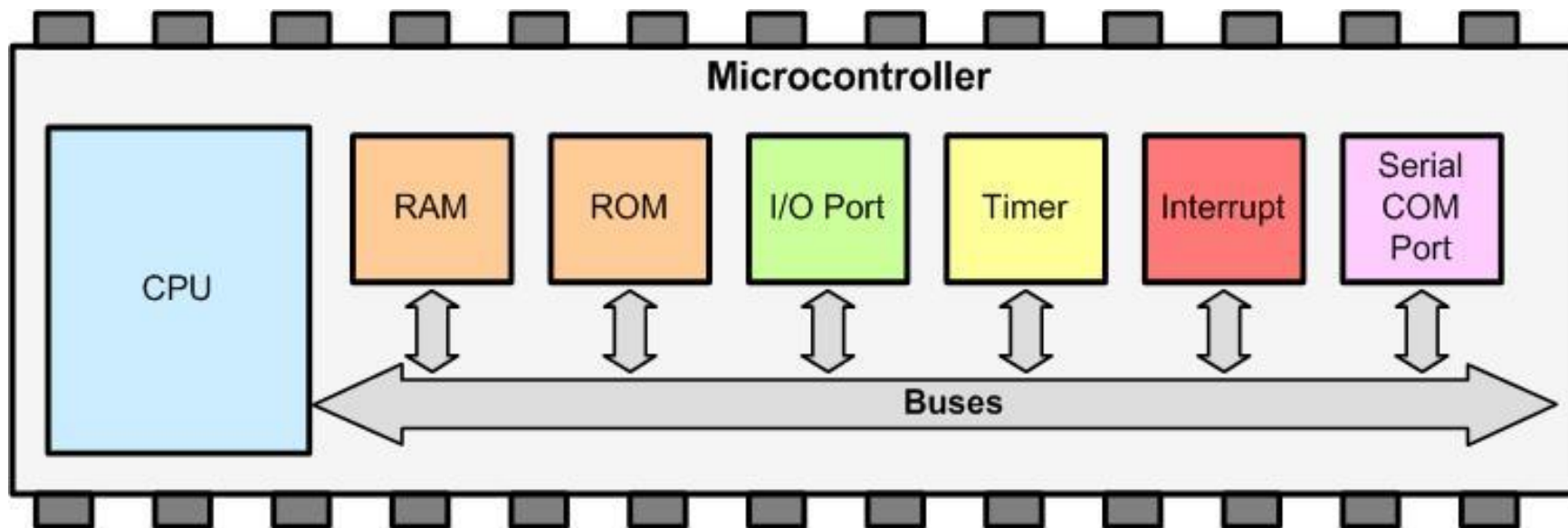
Outlines

- Microprocessors vs Microcontrollers
- The RISC Design
- The ARM Design
- Instruction Set
- RISC vs CISC
- ARM Cortex Variations
- Registers
- Memory Map
- Simple Assembly Code

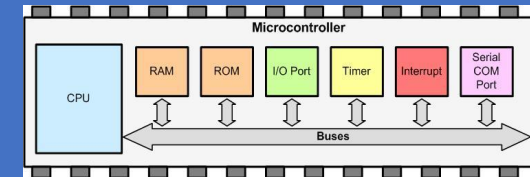
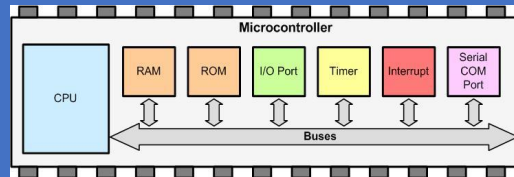
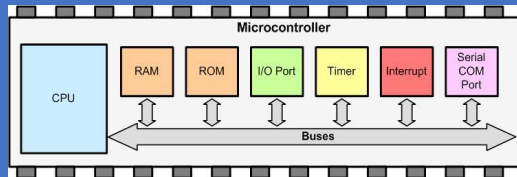
A Computer Made by General Purpose Microprocessor



Simplified View of the Internal Parts of Microcontrollers (SOC)



Embedded System



Instruction Set

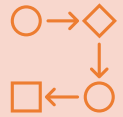
Instruction set = Processor's Language

- Each instruction does a very specific job, like moving data, adding numbers, or controlling the program flow.
- The processor reads and executes these instructions one by one to carry out a program's tasks.

Why is it important?

- Understanding the instruction set is important because it tells you how the computer can understand and perform tasks.
- Different processors might have different instruction sets, and that affects how software is written and how efficiently it runs.

Types of Instruction Sets



RISC – Reduced Instruction Set Computer



CISC – Complex Instruction Set Computer

RISC vs. CISC

CISC

Compiler

Generates
machine code

More complexity

Processor

RISC

Compiler

Generates
machine code

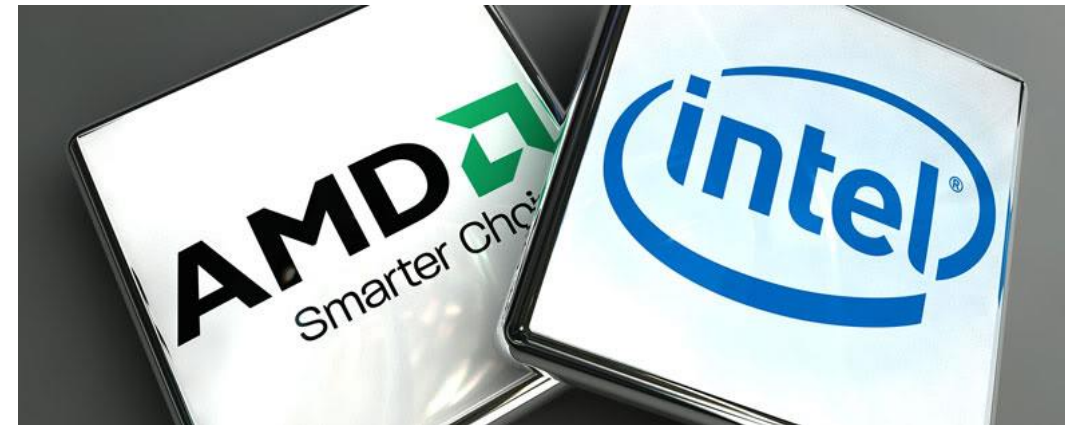
More complexity

Processor

Microprocessor with RISC and CISC

arm

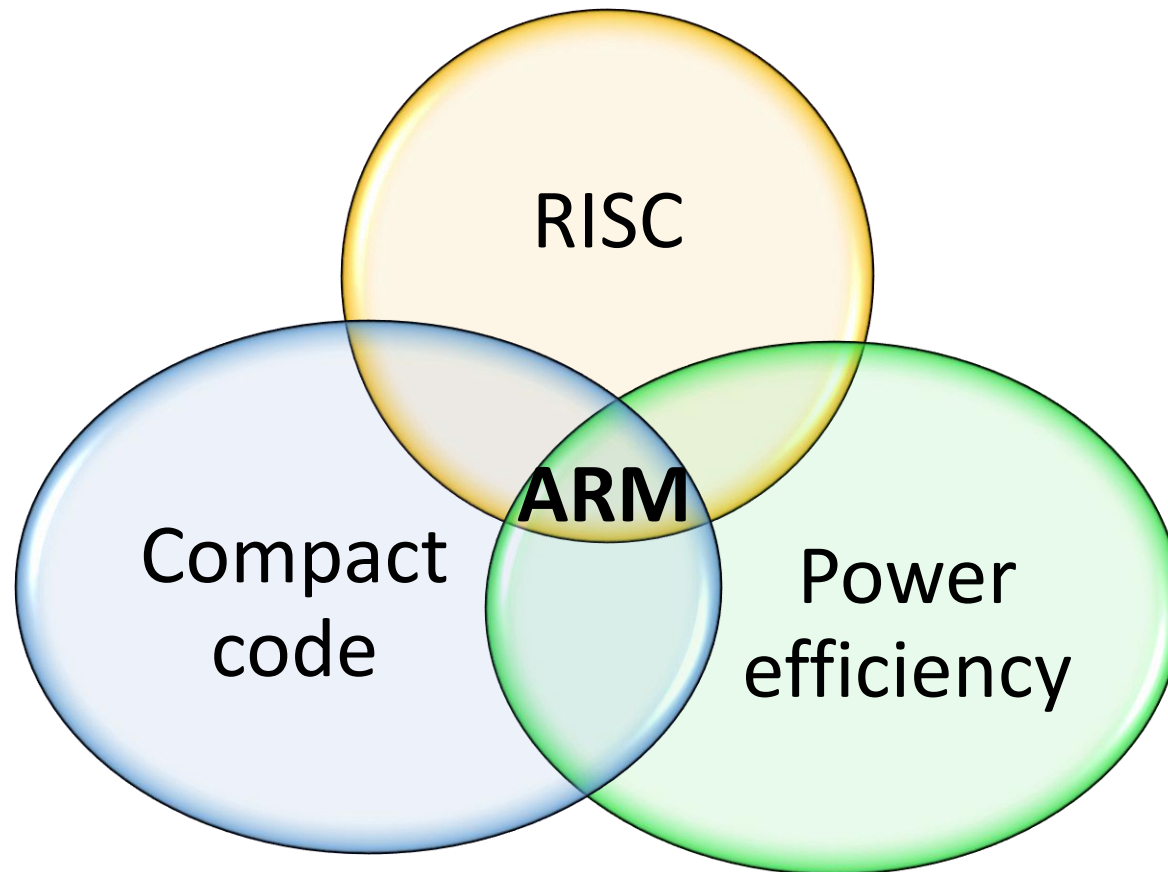
VS.



RISC

CISC

ARM



There are many differences between Intel and ARM, but the main difference is the **instruction set**.

CISC (Complex Instruction Set Computing)

- CISC processor has a **larger, diverse and more feature-rich instruction set** which can lead to shorter programs (less number of instructions per program).
- Complexity may result in **longer instruction cycles**. Programs may be **harder to optimize** for modern superscalar architectures.
- Instructions often **reference memory directly**.
- CISC processors are mainly used in normal PC's, Workstations, and servers.

RISC (Reduced Instruction Set Computing)

- RISC processors have **small, fixed instruction set** (100 instructions or less) with emphasis on simplicity and regularity.
- Simpler instructions enable **faster execution**. It is **easier to optimize programs** due to the simplicity of instructions.
- **Memory access is minimized**, promoting the use of registers.
- ARM microprocessors are **power efficient**.

Use cases

CISC

- **General-purpose desktop systems**

RISC

- **Embedded systems**
- **Mobile devices**
- **Performance-critical applications**

ARM as a Company

- **ARM Holdings plc** is a design firm based in Cambridge, England
- ARM formerly an acronym for Advanced RISC Machines
- ARM **does not** manufacture processors (hardware)
- They come up with **the design of architecture** and they sell the license for architecture to other companies such as Apple, Qualcomm, Texas Instrument and etc.

ARM Key Features

RISC Design:

- Simple, fixed-length instructions
- Emphasis on performance and efficiency

Scalability:

- Wide range of processors for diverse applications
- From low-power microcontrollers to high-performance servers

Energy Efficiency:

- Power-efficient designs for battery-powered devices
- ARM's role in the mobile revolution

Architectural Simplicity:

- Regular and consistent instruction set
- Facilitates easier pipeline design and optimization

Application Examples

Mobile Devices
(Apple and
Android)

Embedded
Systems

IoT (Internet of
Things)

Consumer
Electronics

Automotive
Systems

Networking
Equipment

Wearable
Devices

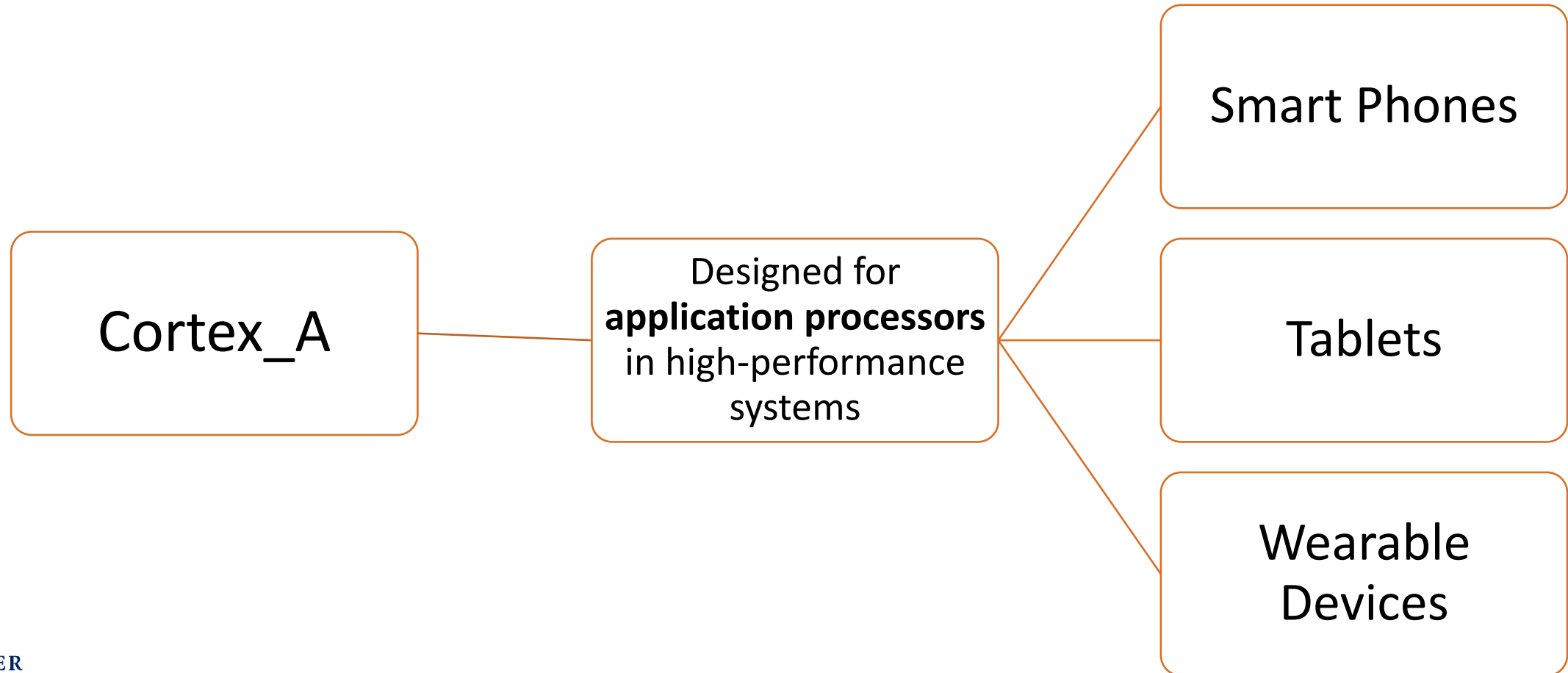
Gaming
Consoles

Aerospace and
Defense

ARM Family and Architecture

ARM FAMILY	ARM Architecture
ARM7	ARM v4
ARM9	ARM v5
ARM11	ARM v6
CORTEX-A	ARM v7-A
CORTEX-R	ARM v7-R
CORTEX-M	ARM v7-M

Cortex-A



ARM Cortex-A



Samsung S8/S9/S10 Smartphones



Apple 6/7/8/X



Samsung smart TVs

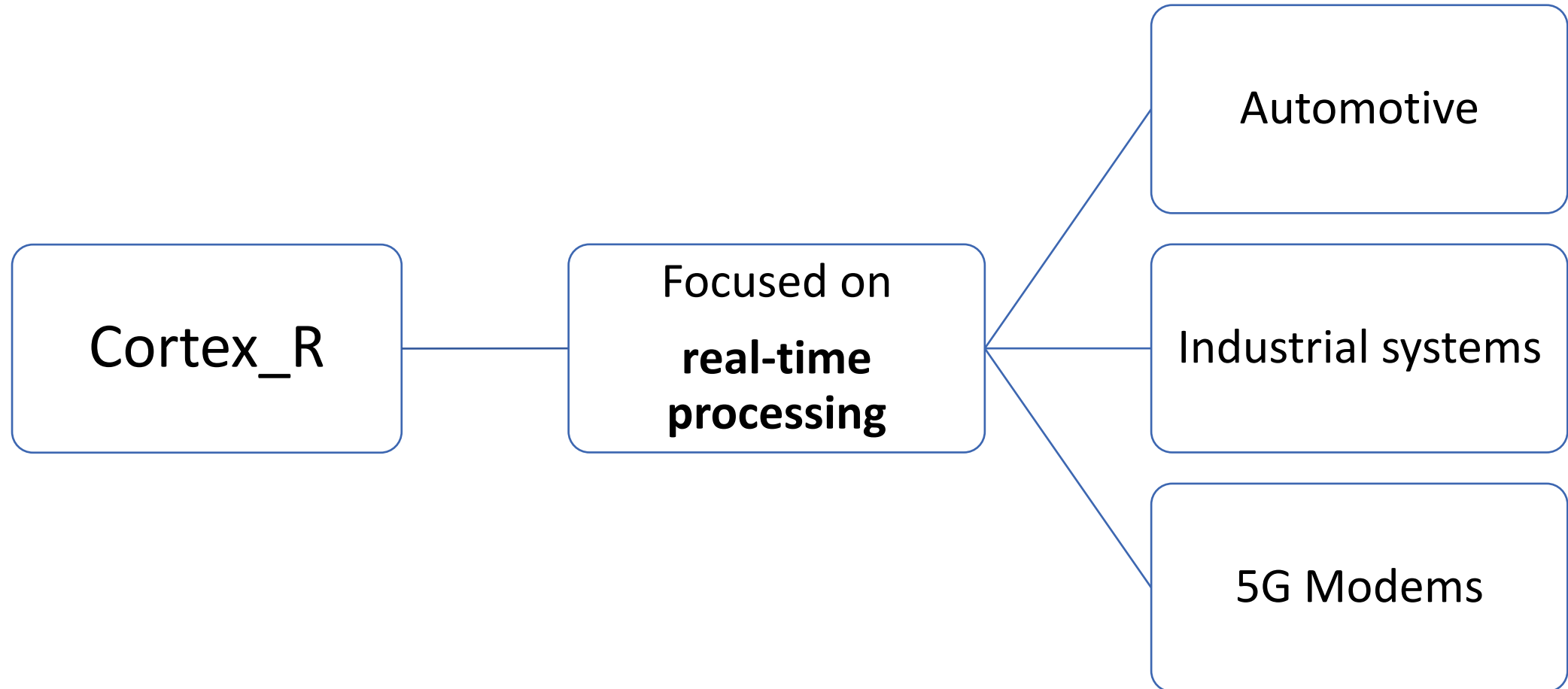


Raspberry Pi 3/3B+/4/5

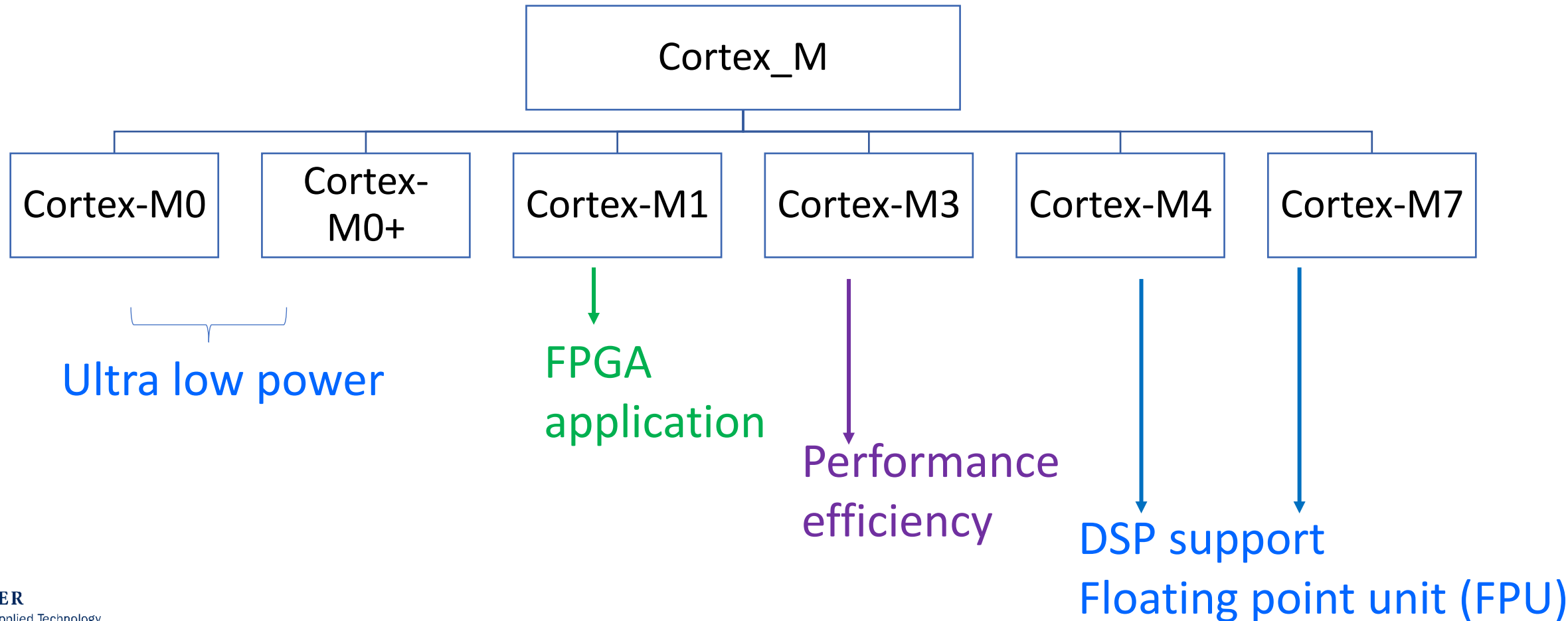


Fossil and Garmin Stopwatches

ARM Cortex-R



ARM Cortex-M



ARM Cortex-M Microcontroller

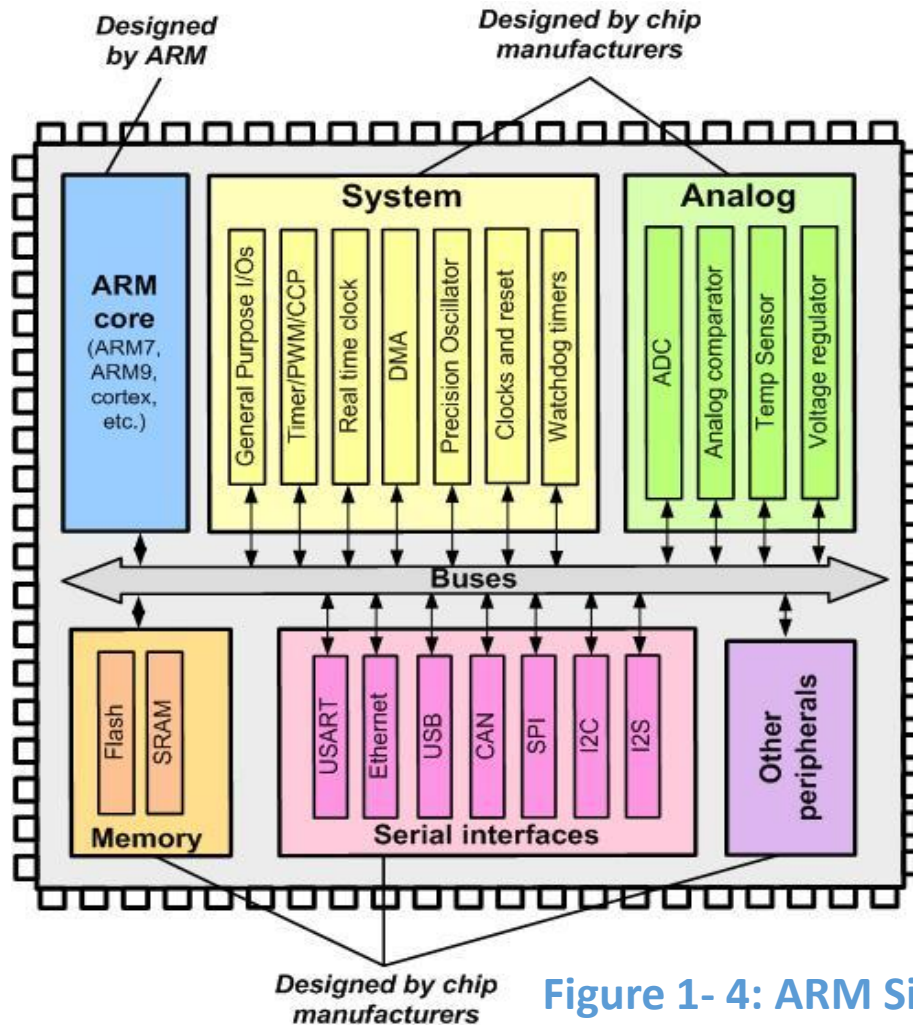


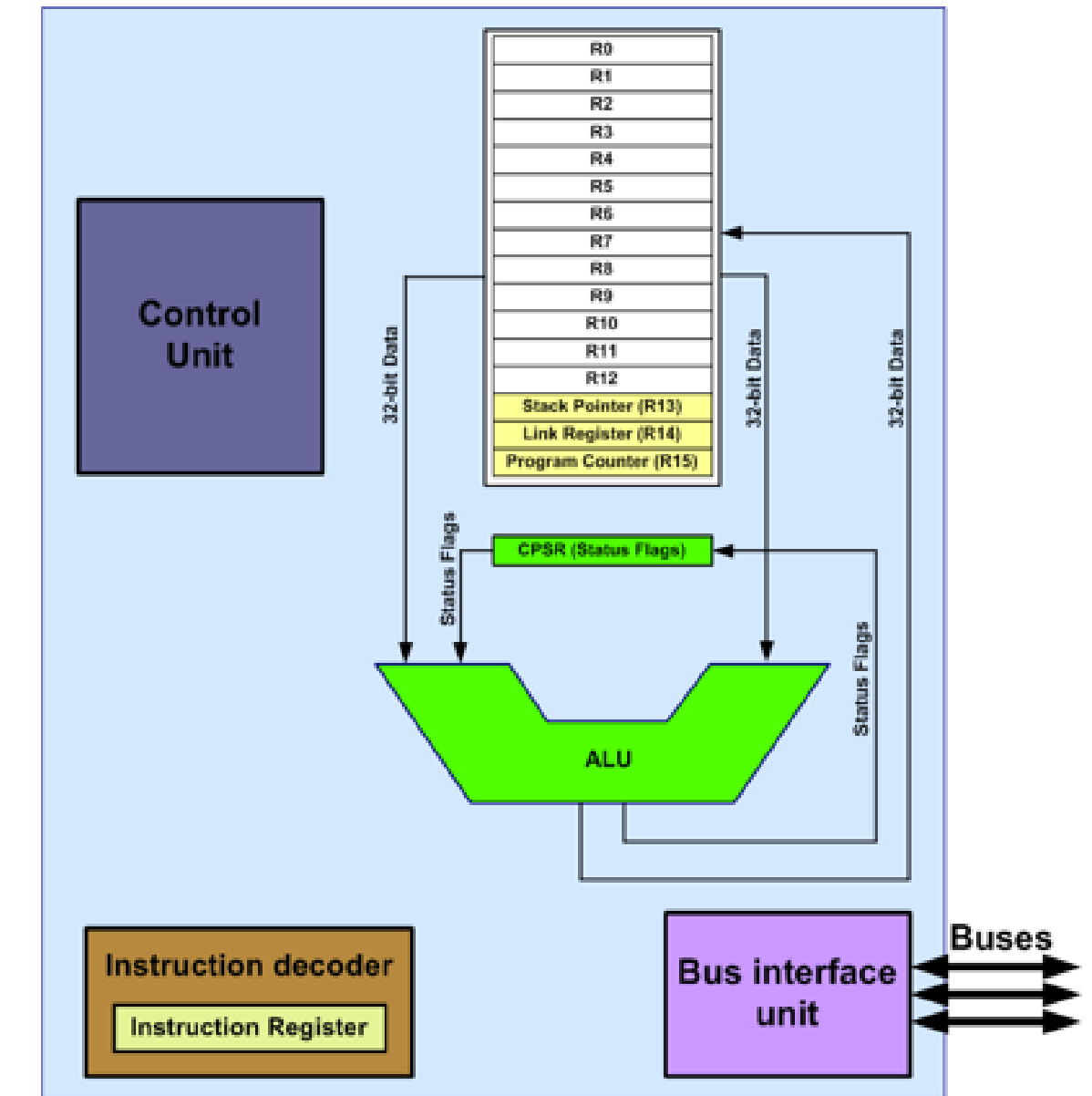
Figure 1- 4: ARM Simplified Block Diagram

Ref: ARM Assembly Language Programming & Architecture by Mazidi, et al.

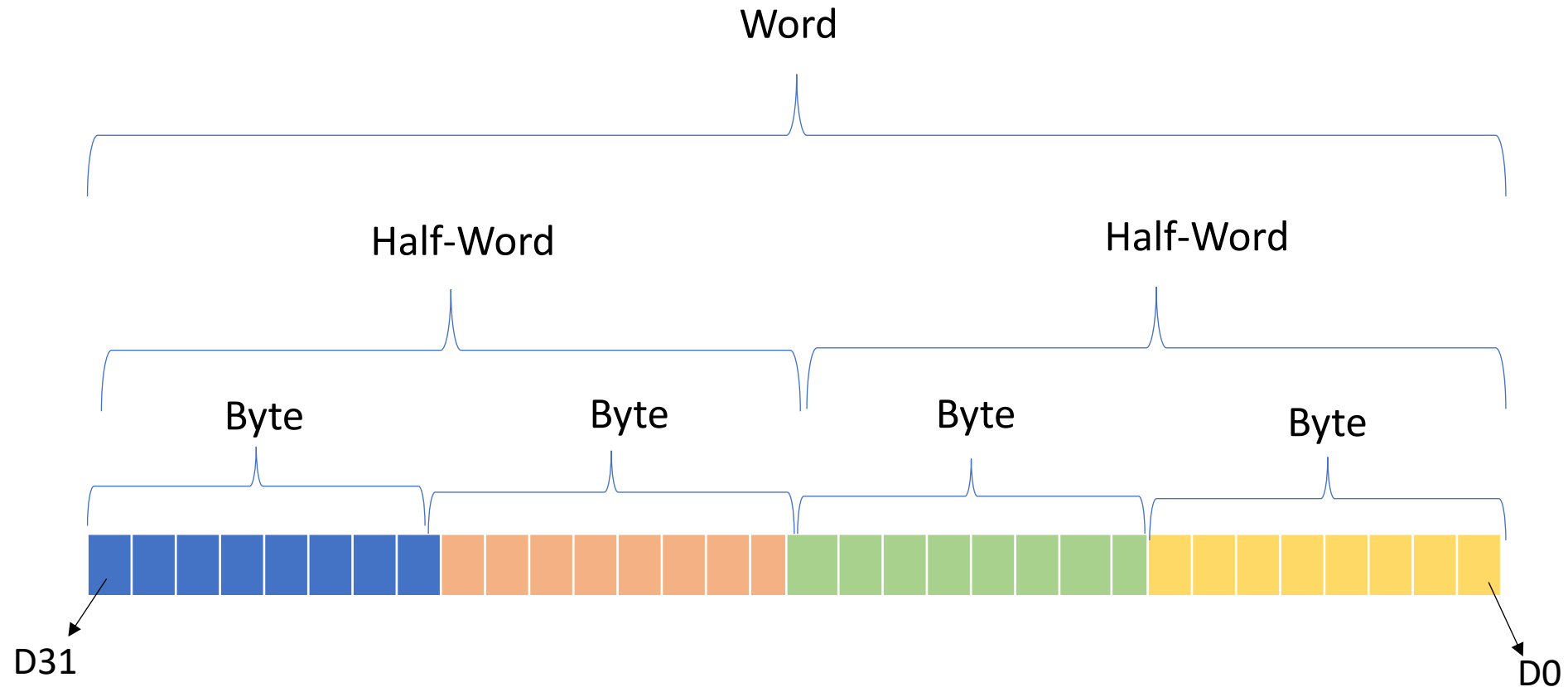
ARM Cortex-M is a 32-bit microprocessor

- **Internal Registers** in the register bank are 32-bits
- **Data Path** is 32-bits
- **Bus Interface** is 32-bits

ARM Cortex-M CPU



General Purpose Registers in ARM (R0-R12)

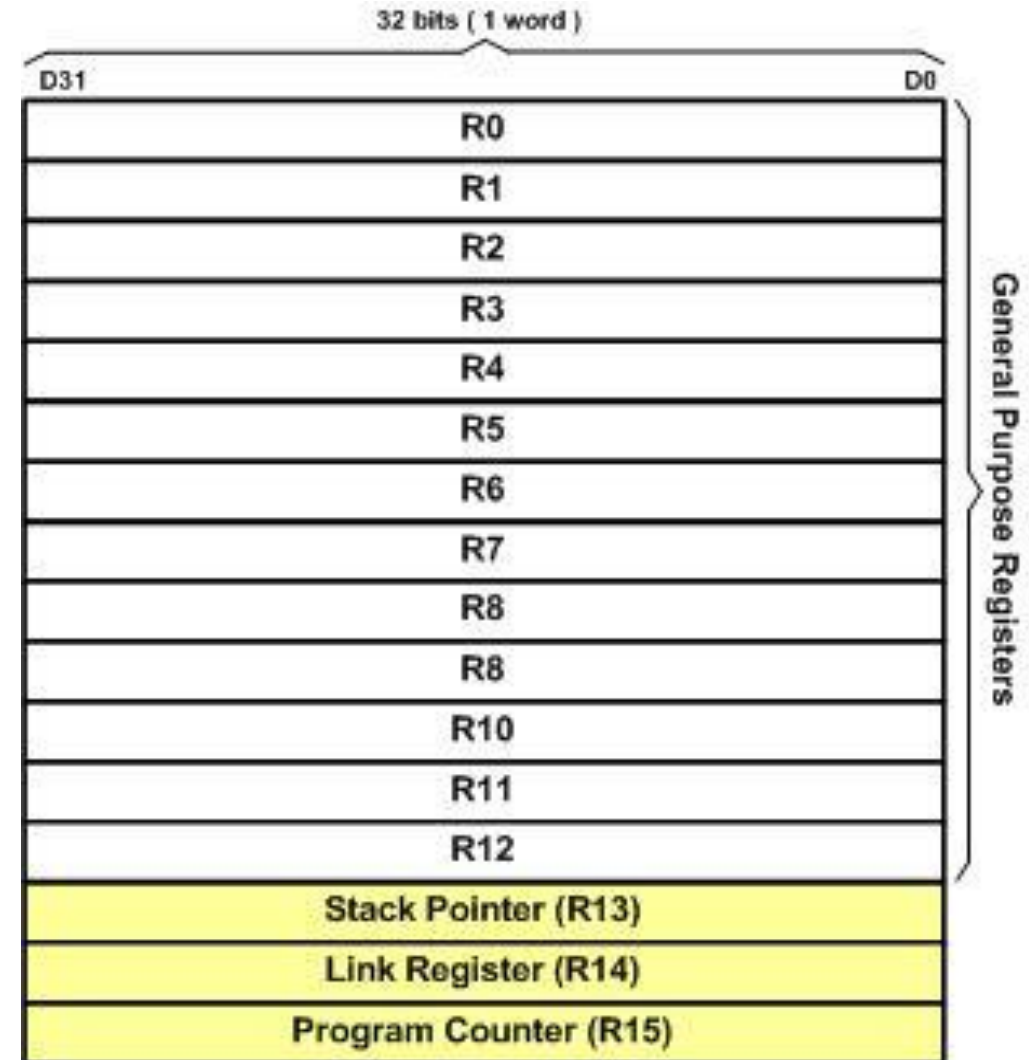


32-bit Register

ARM Registers

Registers are 32-bit storage units
inside CPU.

The number of registers
depends on the ARM version.



ARM Assembly Language Programming & Architecture by Mazidi, et al.

Special Function Registers in ARM

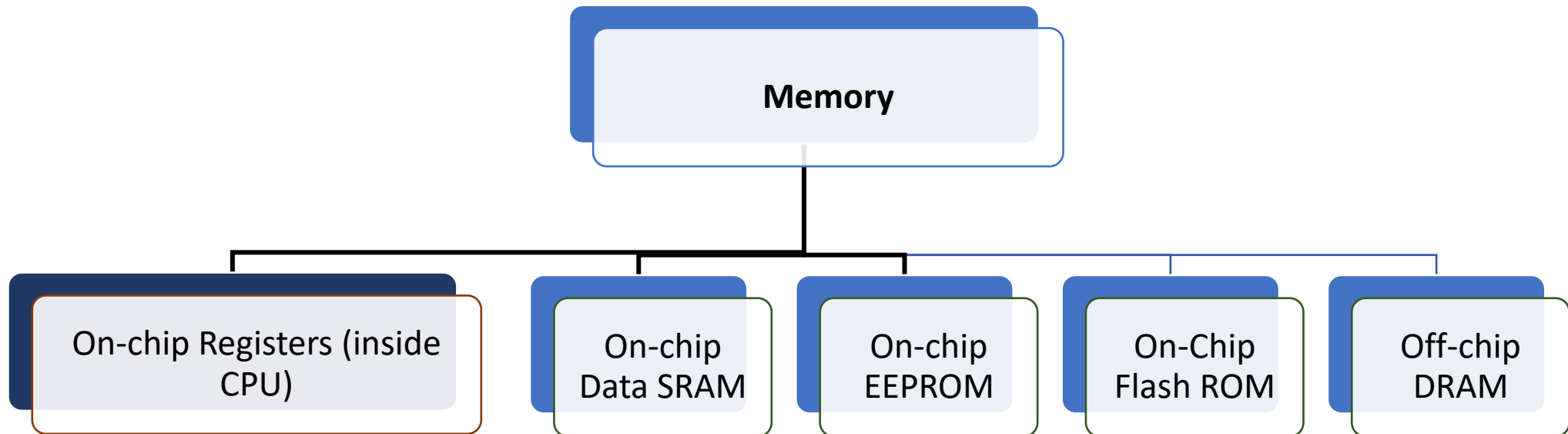
In ARM the R12, R14, and R15 and CPSR (current program status register) registers are called SFRs (special function registers) since each one is dedicated to a specific function.

R13: stack pointer (SP)

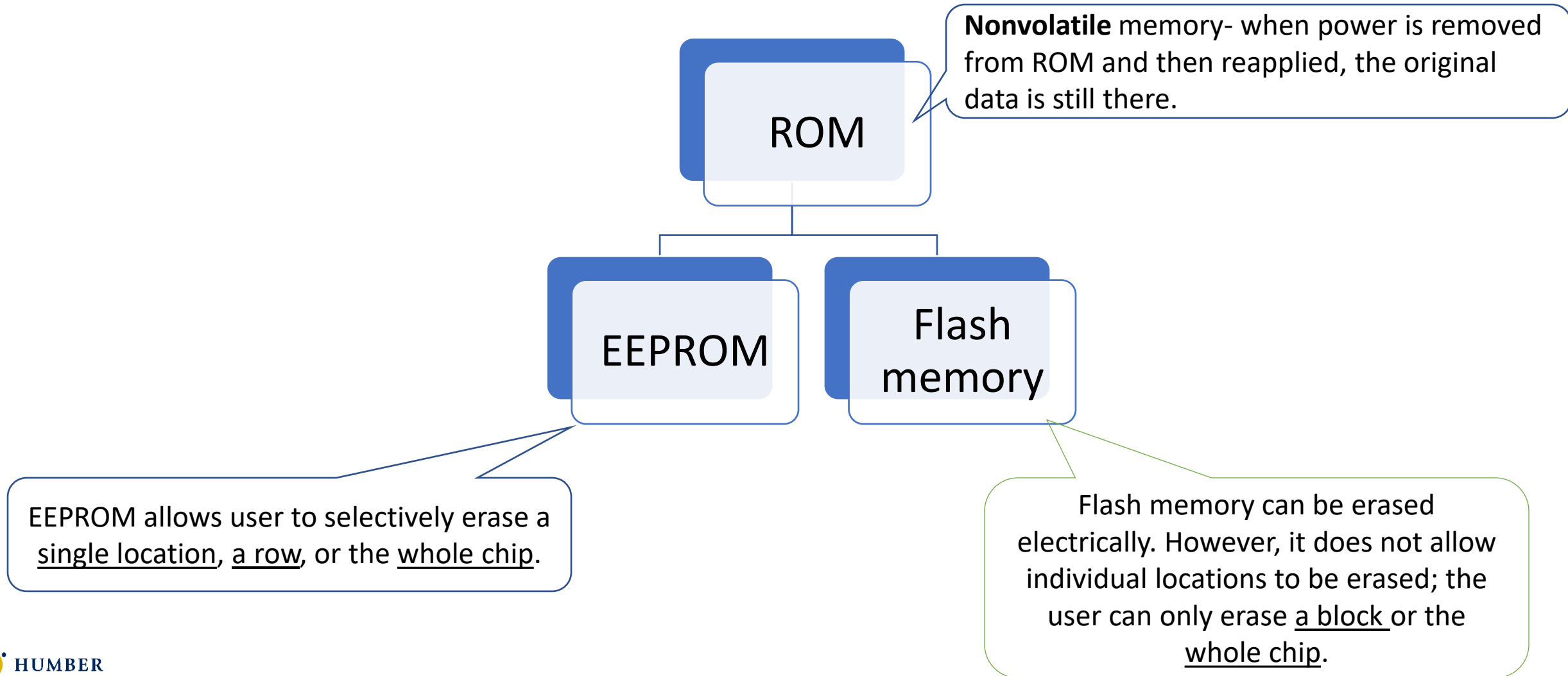
R14: link register (LR)

R15: program counter (PC)

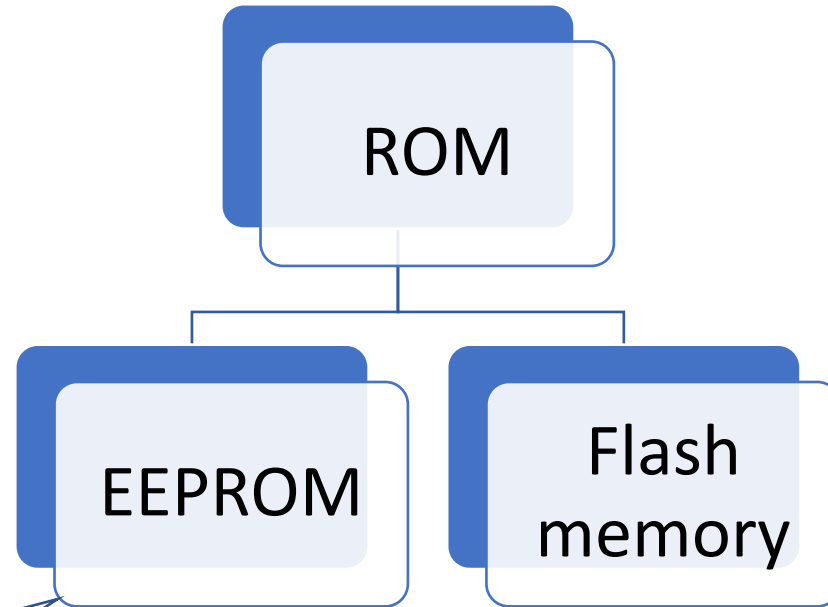
Memory Types in ARM Microcontrollers



Read Only Memory (ROM)



Read Only Memory (ROM)



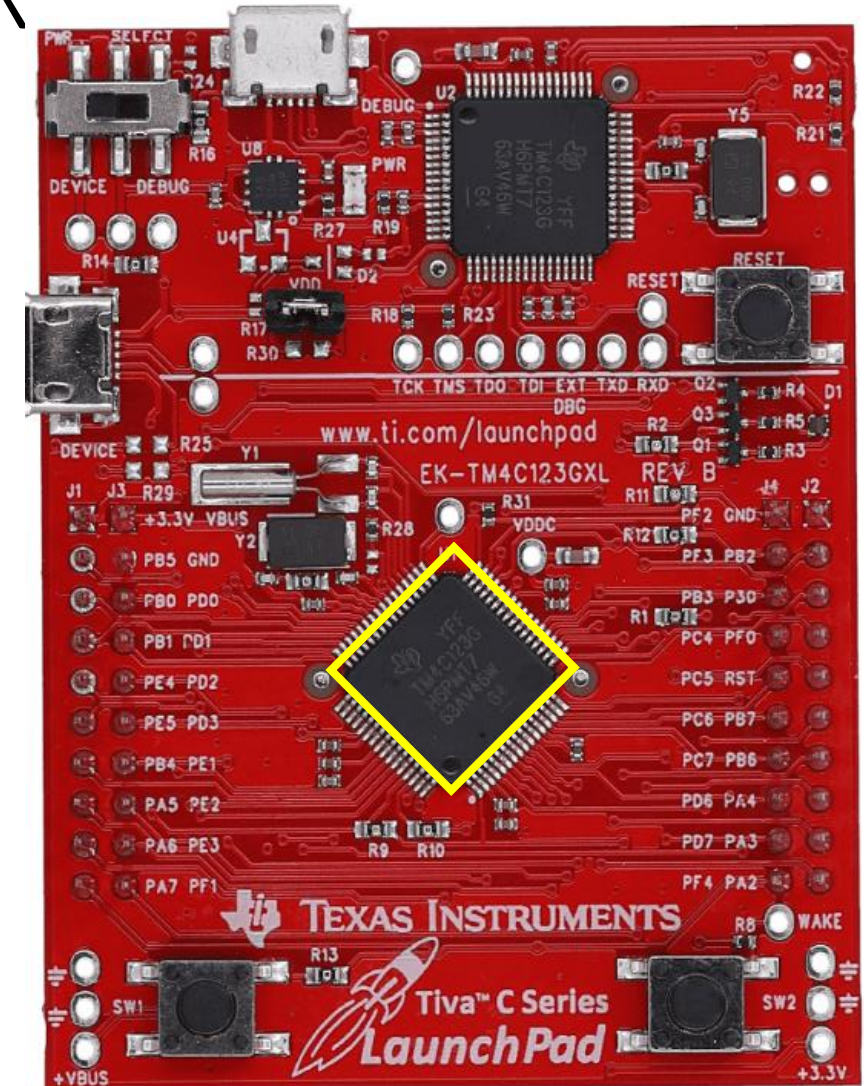
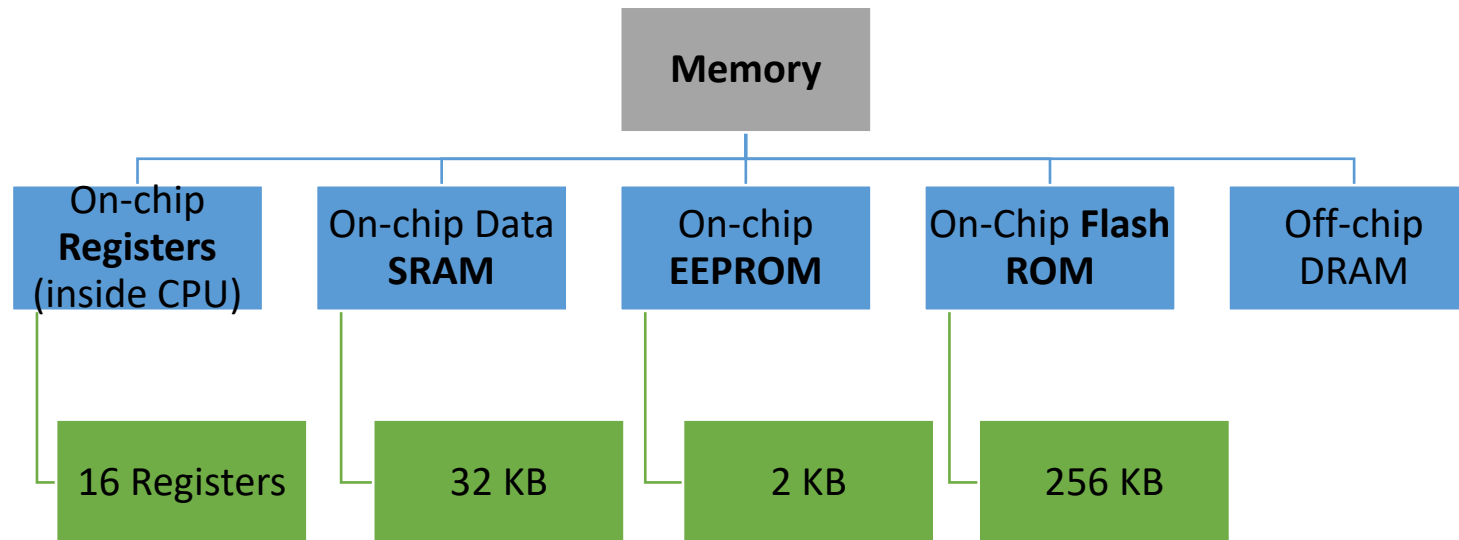
EEPROM is **slower** compared to flash memory. It is used to store data that needs to remain unchanged when power is removed and reapplied.

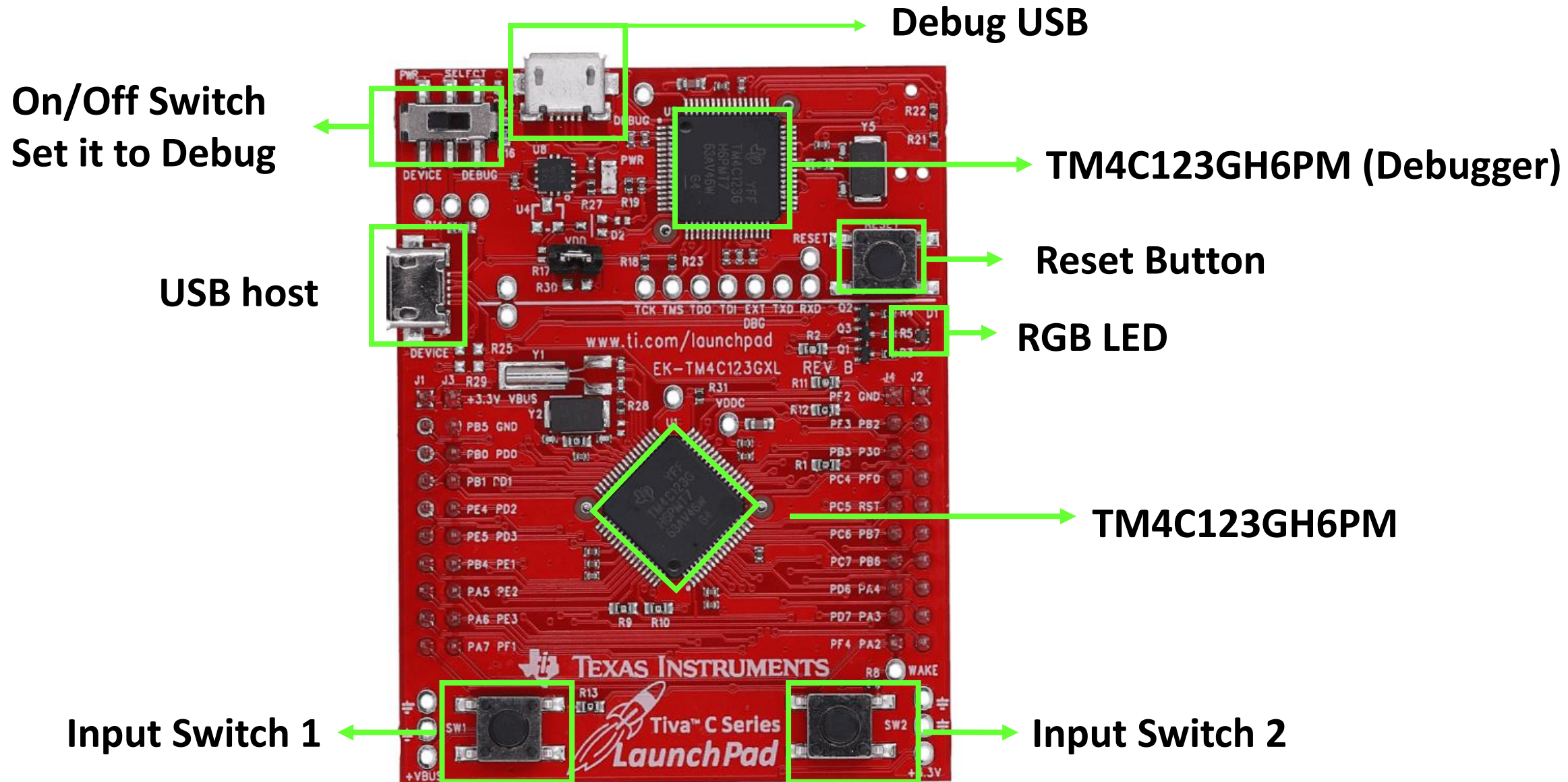
Flash memory is **faster** compared to EEPROM and therefore used to store our program (firmware) in MCU.

Memory Space Allocation in the ARM

Memory Type	Program Use
Flash	All program code (Firmware)
EEPROM	1. For storing critical data 2. Critical Constants such as messages and lookup tables 3. Nonvolatile variables (database)
RAM	1. Data variables and stack 2. Accessible by microcontroller instructions
On-chip Peripheral and I/O Registers	1. Dedicated to registers of peripherals such as timers, serial communication, ADC and so on. 2. The function and location of each register is fixed by chip vendor at the time of design.
Off-chip DRAM	1. Can be implemented for external memory connection

TIVA C LaunchPad TM4C123GX1





TM4C123GH6PM Microcontroller

- CPU: Cortex-M4F
- Microcontroller has 64 Pins.
- Data type: Cortex-M4F supports **32-bit word, 16-bit half word and 8-bit bytes.**
- All instructions and data memory access are **little endian.**
- Bus interface: 32-bit

What does it mean to have a 32-bit Bus Interface?

x-bit $\rightarrow 2^x$ combination

With 8-bit interface bus we have $2^8 = 256$. If we use each number as an address of a memory location, we can have 256 addressable memory locations.

With 32-bit interface bus, we have **$2^{32} = 4294967296 \sim 4\text{Gbytes}$** of addressable memory.

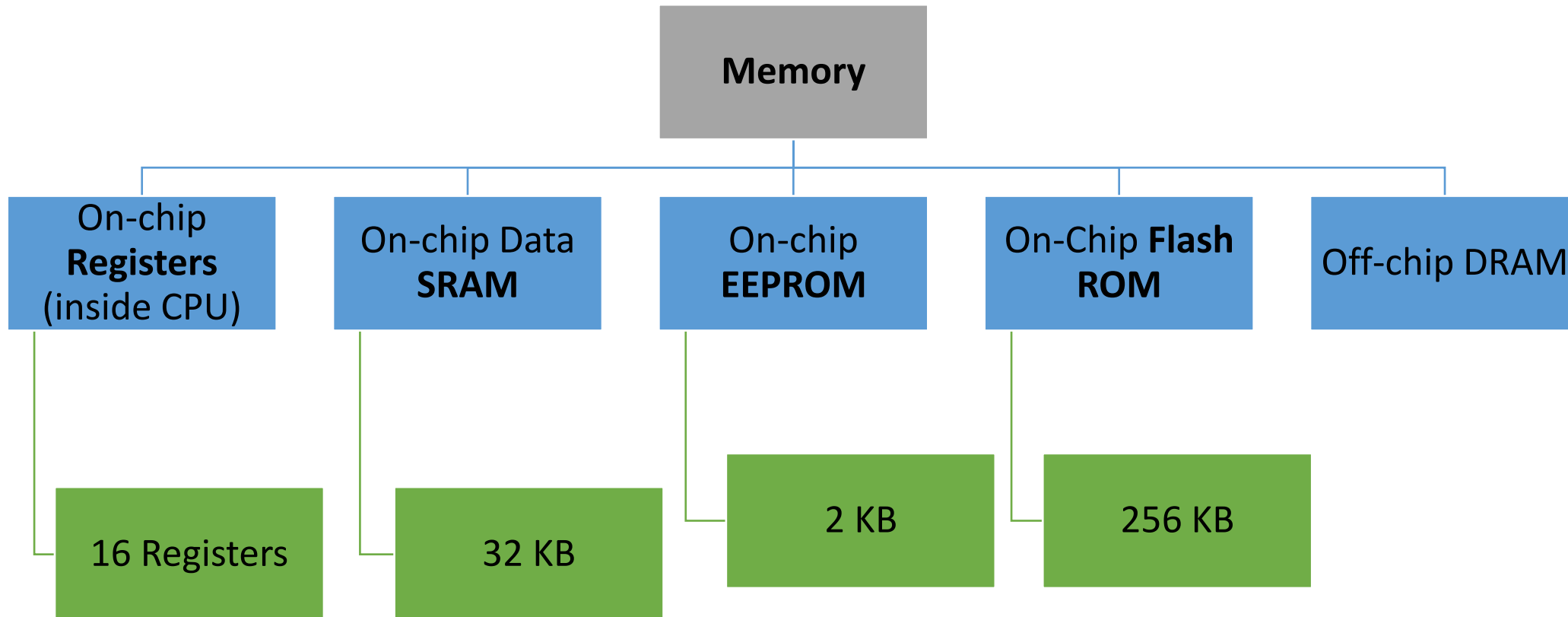
Memory Model

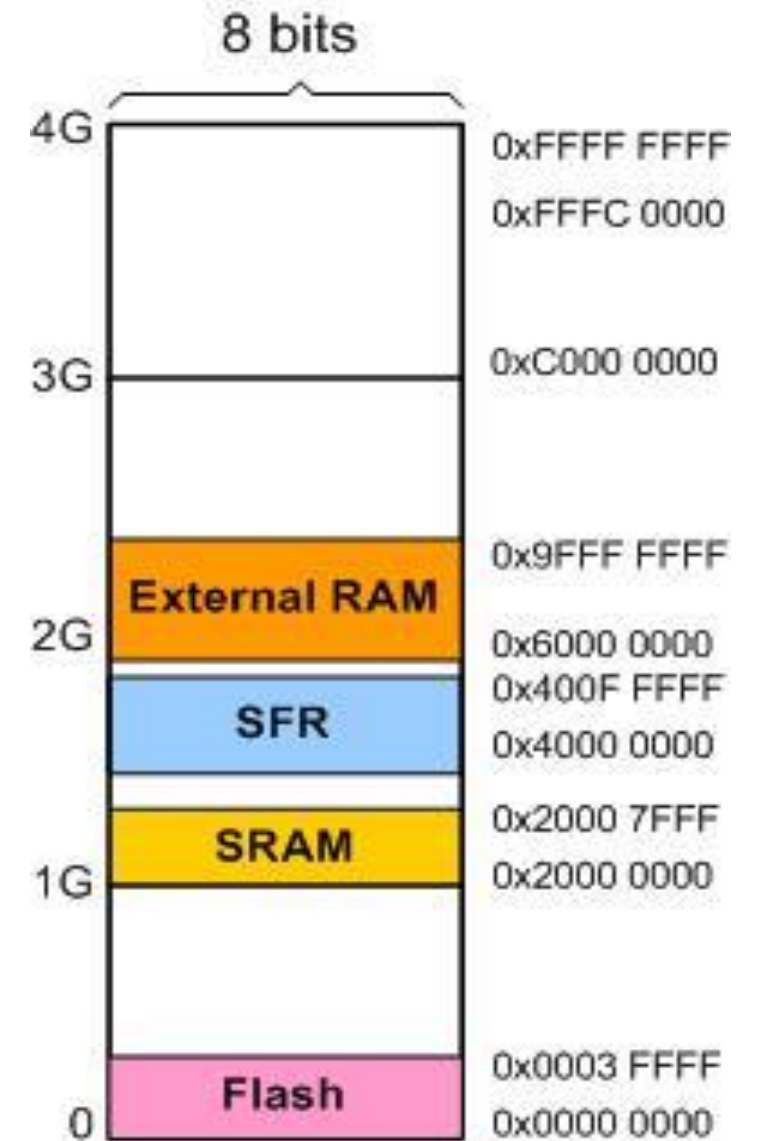
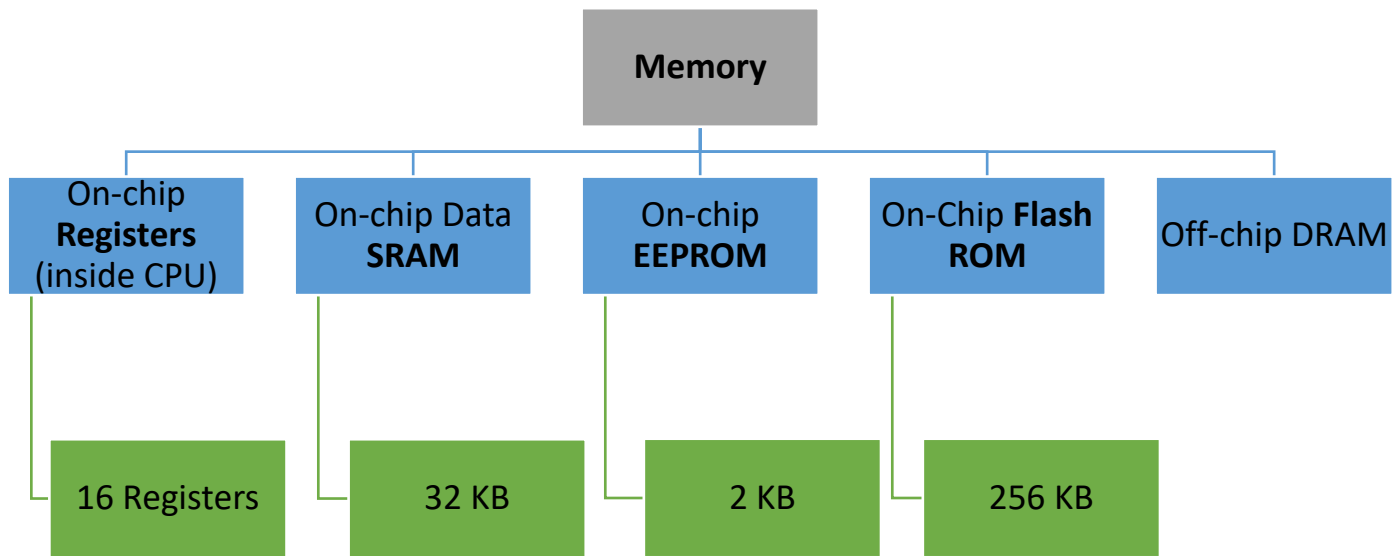
4294967295 =0x FFFF,FFFF

.
.
.

1 =0x 0000,0001

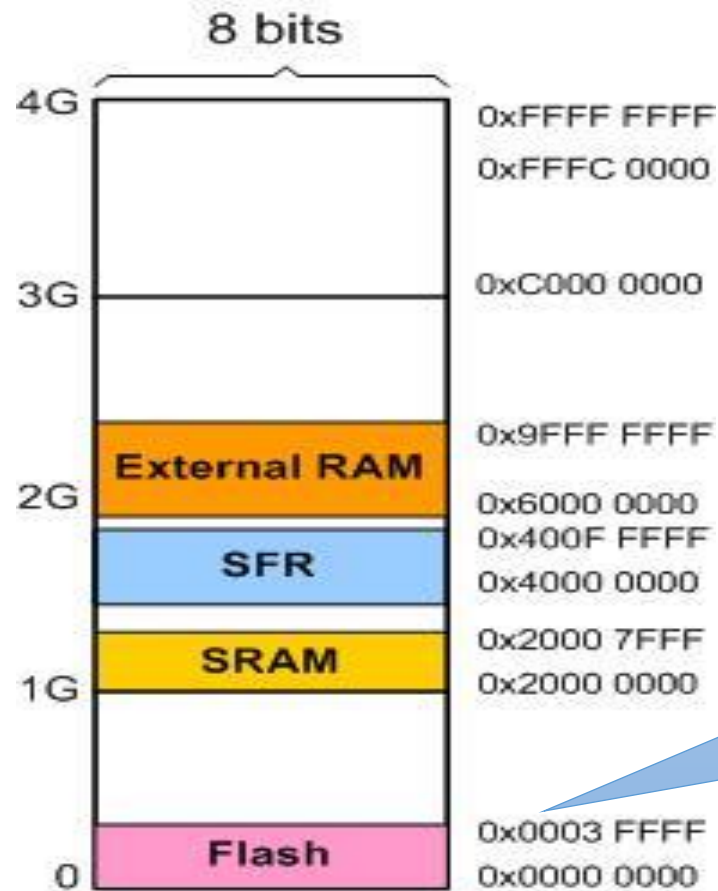
0 =0x 0000,0000





Texas Instruments Tiva TM4C123GH6PM Microcontroller

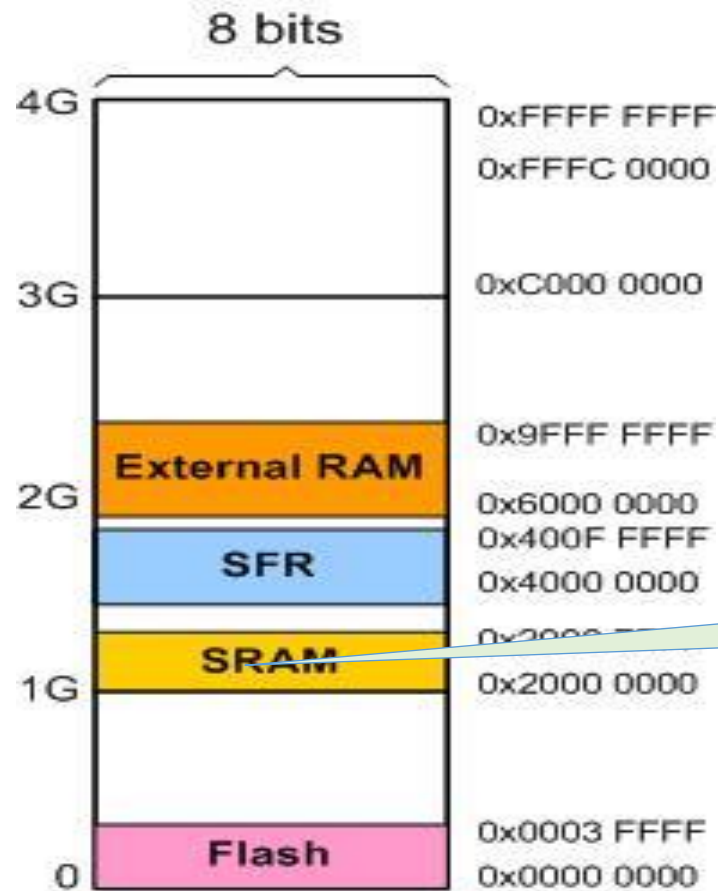
(Memory Map)



For **program code**.
Organized in 1-KB block. Each
block can be independently
erased and written to.

Texas Instruments Tiva TM4C123GH6PM Microcontroller

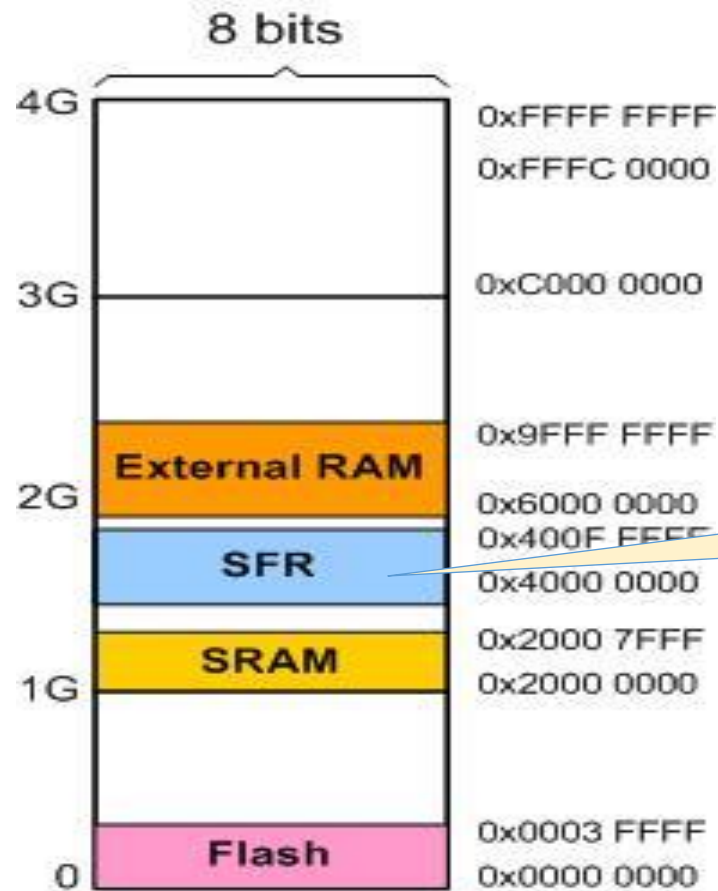
(Memory Map)



For variables and stack.

Texas Instruments Tiva TM4C123GH6PM Microcontroller

(Memory Map)

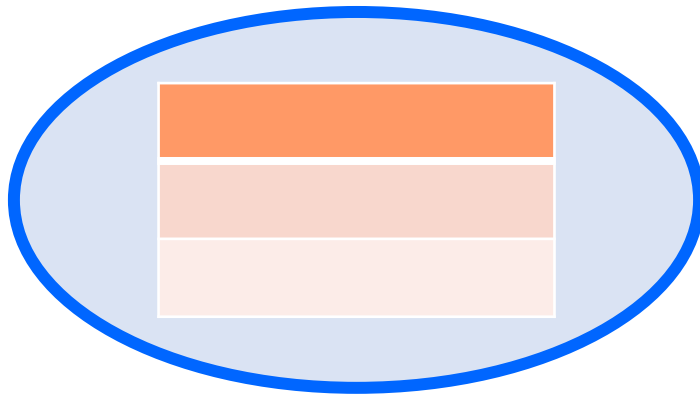


For peripherals such as I/O, Timer, ADC, etc.
Refer to Table 2-4 in [TMC123GH6PM Datasheet](#)

Programming Microcontrollers

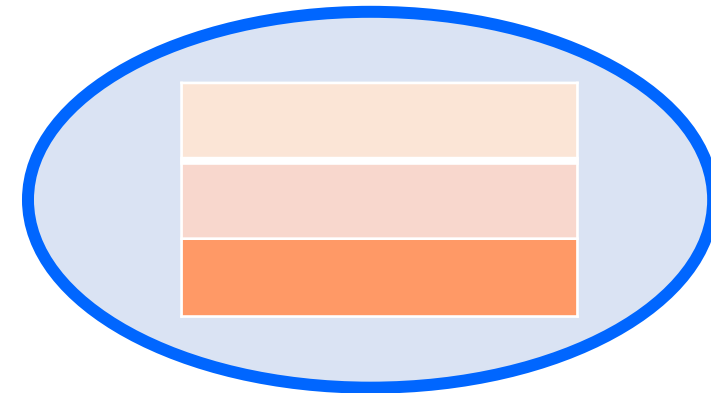
High-level Programming (C/C++)

- Hides details from programmer via layer of abstraction

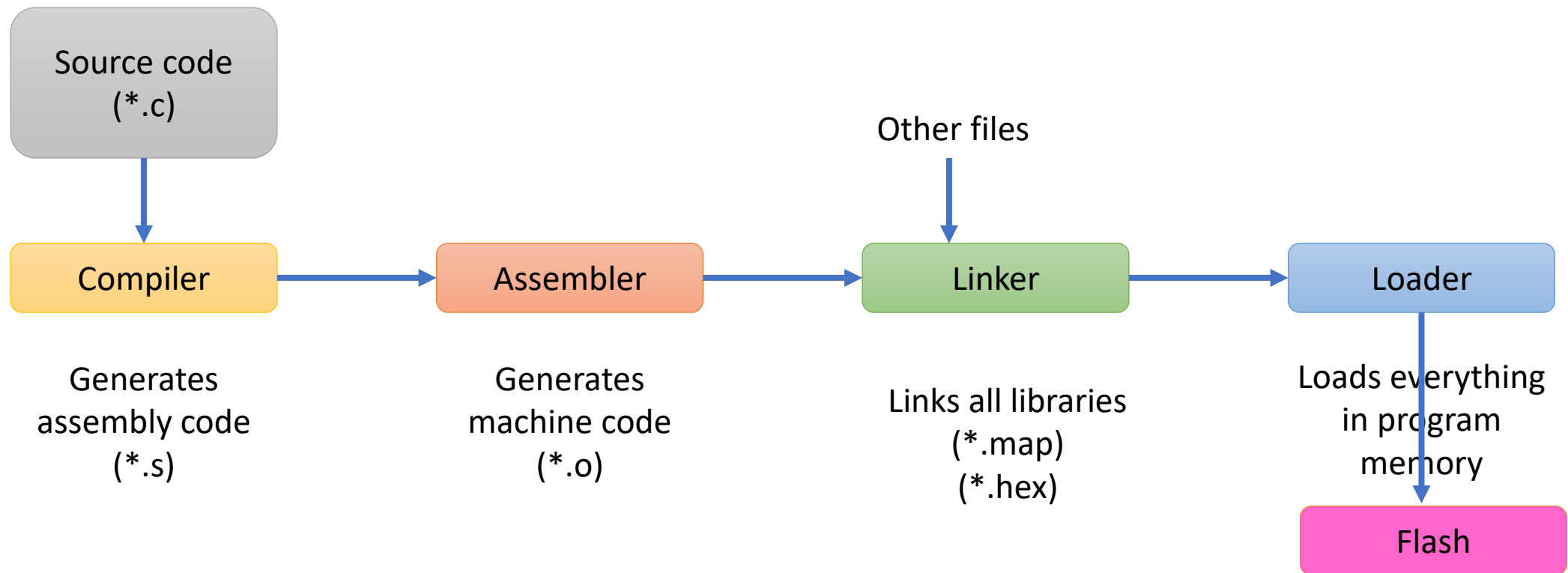


Low-level Programming (Assembly)

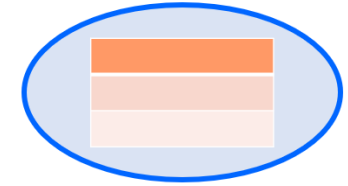
- Aligns closely with processor instructions



Creating an Assembly Program and Execution



High-level Programming (C/C++)



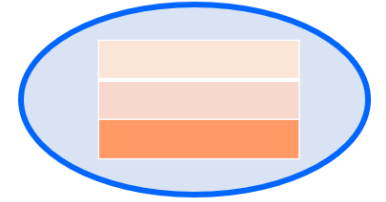
Advantages ✓

- Faster development
- Codes are better portable from one platform to another one
- Easier to read, maintain and reuse codes

Limitations

- Less control over optimizing codes
- Longer compiling time
- Higher level of abstractions

Low-level Programming (Assembly)



Advantages ✓

- Codes are written closer to machine language
- Less compiling time
- Better control over optimizing code
- Memory efficiency

Limitations

- Increased development time
- Harder to read, maintain and reuse codes
- Limited portability from one platform to another

ARM Assembly Language Syntax

A common format for assembly instruction in ARM CPU is:



Instructions tell **CPU** what to do!

ARM Assembly Language Syntax

A common format for assembly instruction in ARM CPU is:

Label	Opcode	Operand	; Comment
Init	MOV	R1, #100	; R1= 100

- **Label (Optional):**

- Used to find the position of the current instruction in memory.
- It helps in **branching** or **jumping** to other parts of the program.

- **Opcode:**

- The **opcode** specifies the operation the processor must perform.
- For example, **MOV** is an opcode to move data, **ADD** is for adding numbers.

- **Operand:**

- The operand tells the processor where to **find the data (source)** and where to **store the result (destination)**.
- For example, in MOV R0, R1, R0 is the destination, and R1 is the source.

- **Comment (Optional):**

- Comments are used to explain or clarify the code.
- They are not executed by the processor. Comments are typically written **after a ; symbol**

Assembler Directives

- Assembler directives **give directions to the assembler.**
- The directives help us develop our program easier and make our program more readable.
- Some widely used assembler directives:

Directive	Description
AREA	Instructs the assembler to assemble a new code or data section
END	Informs the assembler that it has reached the end of the source code
EQU	Associates a symbolic name to a numeric constant
INCLUDE	It adds the content of a file to the current program
EXPORT	make a symbol (such as a label, variable, or function) visible or accessible to other modules or programs during the linking process .

AREA

- The AREA directive tells the assembler to define a new section of memory.
- The memory can be:
 - code (instructions) or data
 - Can have attributes such as READONLY, READWRITE, and so on
- Every assembly language program has at least one AREA.
- Format:

AREA section_name, attribute, attribute, ...

Examples for AREA directive

- Define a new section for code:

```
AREA      MY_PROGRAM, CODE, READONLY
```

- Define a new section for data, variables:

```
AREA      My_VARIABLES, DATA, READWRITE
```

- Define a new section for constant values in flash memory:

```
AREA      My_CONSTANTS, DATA, READONLY
```

EQU

- This is used to define a constant value or a fixed address by a name to make the program easier to read.
- The EQU directive does not set aside storage for a data item in the program, it just associates an identifier with the constant value.
- Example:

```
COUNT EQU 25  
  
...  
MOV R2, #COUNT ; R2 = 0x25
```

Assembler data allocation directives

- In most assembly languages there are some directives to allocate memory and initialize its value.

Directive	Description
DCB	Allocates one or more bytes of memory and defines the initial runtime contents of the memory
DCW	Allocates one or more halfwords of memory (two bytes) and defines the initial runtime contents of the memory
DCD	Allocates one or more words of memory (four bytes) and defines the initial runtime contents of the memory

EXPORT

- **Makes symbols available across modules:**
 - When you write large programs, they are often divided into different modules (files).
 - The **EXPORT** directive allows symbols (such as functions, variables, or labels) defined in one module to be used in other modules.
- **Linking:**
 - During the **linking phase** of the program compilation, the **EXPORT** directive ensures that the symbol is **exposed to the linker**, allowing it to be linked properly with other parts of the program.

Simple Assembly Instructions

- MOV (move a value to a register)
- LDR (loading 32-bit values)
- Arithmetic Operations
 - ADD
 - SUB
 - AND

MOV (move)

Move data within processor without accessing memory.

Syntax

```
MOV Rd, #K
```

; d = 0 to 15

; K is an 8-bit value

; Rd = K

Examples

```
• MOV R5, #53
```

```
; R5 = 53
```

```
• MOV R9, #0x23
```

```
; R9 = 0x23
```

MOV (move)

Syntax

```
MOV Rd, Rs
```

; d and s = 0 to 15

; Rd = Rs

; Rd and Rs are both
registers.

Examples

- MOV R5, R2

; R5 = R2

- MOV R9, R8

; R9 = R8

Important notes about MOV instruction

1. Immediate Value Syntax:

- The **#** sign is used to specify an immediate value (a constant value) in ARM assembly.

2. Hexadecimal Values:

- The **0x** prefix indicates that the value following it is in **hexadecimal** (base 16).
- For example, 0x55 means 55 in hexadecimal notation, which is equal to 85 in decimal.

Important notes about MOV instruction

3. MOV Instruction Behavior:

- When using the MOV instruction to load an immediate value

e.g., MOV R1, #0x55:

- The 8-bit value (in this case, 0x55) will be moved into the 32-bit register (e.g., R1).
- The remaining 24 bits in the register are filled with zeros, resulting in a 32-bit value with the form 0x00000055 in the register.

Exercise

- Write a line of code to load R11 with the value of R2.
- Write a line of code to load R7 with the value of 39 (decimal).
- Write a line of code to load R3 with the value of 87 in hexadecimal.

LDR

1. Loads a register with a 32-bit value

Syntax

```
LDR Rd, =K
```

; K is a 32-bit value

; Rd = K

Example

```
LDR R5, = 2324  
; R5 = 2324
```

```
LDR R7, = 0x54021365  
; R7 = 0x54021365
```

LDR

2. Loads a register with the **content of a memory** located pointed to by another register

Syntax

```
LDR Rd, [Rx]
```

;d, x an index from 0 to 15

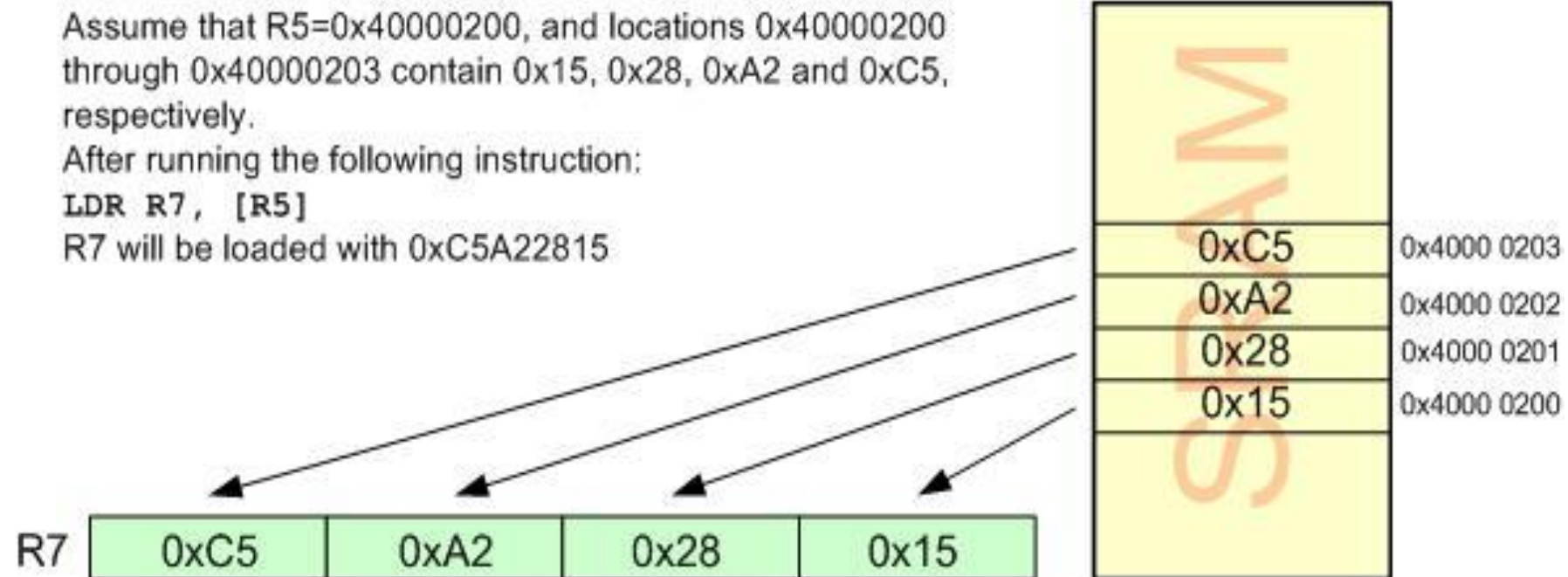
Example

```
LDR R4, =40000200
```

```
LDR R5, [R4]
```

Look at the next
slide ->

Figure 2-5: Executing the LDR Instruction



Store Instruction in ARM

Syntax:

```
STR Rx, [Rd]
```

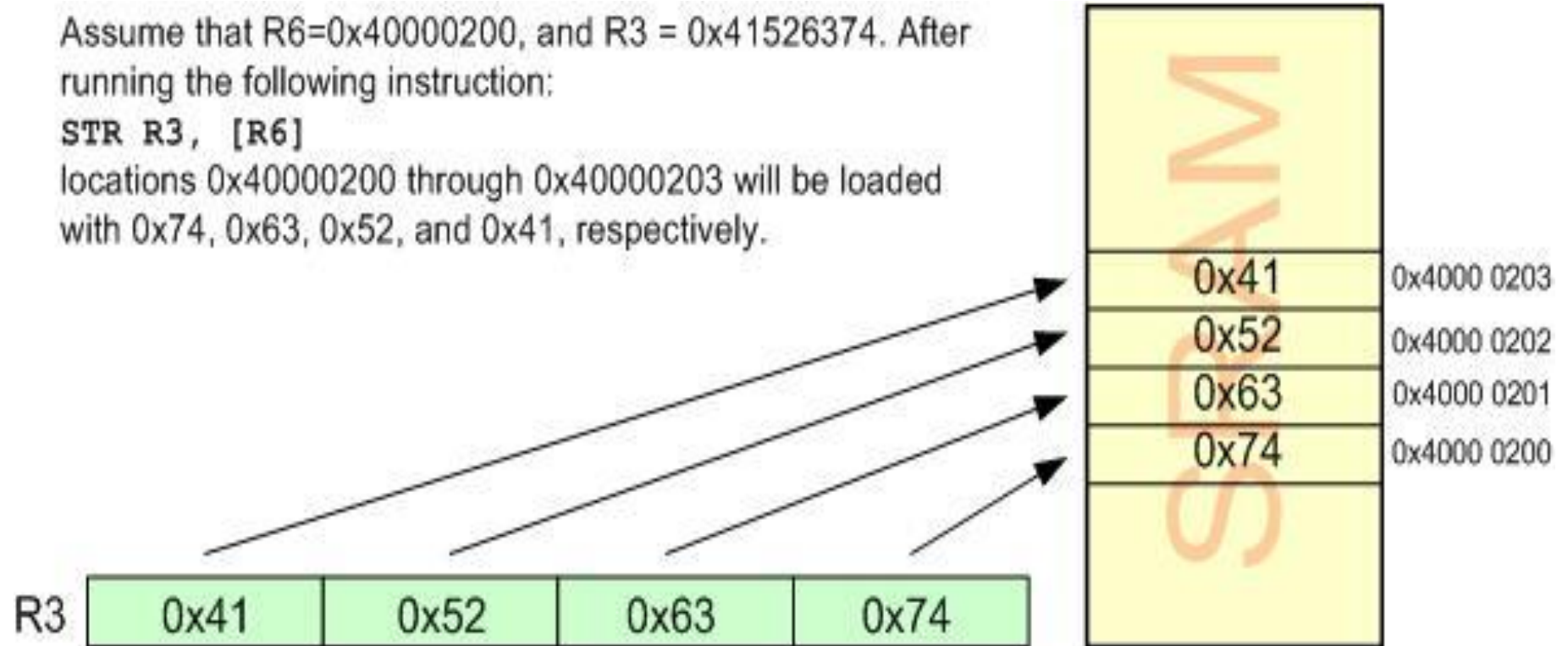
; x, d are index number
between 0 and 15

Store the content of register x into
locations pointed by Rd

```
LDR R6, =40000200  
LDR R3, =41526374  
STR R3, [R6]
```

Look at the next
slide ->

Figure 2-6: Executing the STR Instruction



Arithmetic Operations, ADD, SUB, AND, etc

Syntax

Instruction Rd, Rn, Op2

; Instruction can be ADD,
AND, SUB

; Add/SUB/AND/ Op2 to Rn
and store the result in Rd

; Op2 can be immediate
value or register Rm

Example

```
ADD R1, R2, #0x23  
; R1 = R2 + 0x23
```

```
ADD R1, R2, R3  
; R1 = R2 + R3
```

```
SUB R5, R1, R7  
; R5 = R1 - R7
```

```
AND R11, R8, #0x53  
; R11 = R8 AND 0x53
```

Instruction		Description
ADD	Rd, Rn,Op2 *	ADD Rn to Op2 and place the result in Rd
ADC	Rd, Rn,Op2	ADD Rn to Op2 with Carry and place the result in Rd
AND	Rd, Rn,Op2	AND Rn with Op2 and place the result in Rd
BIC	Rd, Rn,Op2	AND Rn with NOT of Op2 and place the result in Rd
CMP	Rn,Op2	Compare Rn with Op2 and set the status bits of CPSR**
CMN	Rn,Op2	Compare Rn with negative of Op2 and set the status bits
EOR	Rd, Rn,Op2	Exclusive OR Rn with Op2 and place the result in Rd
MVN	Rd,Op2	Store the negative of Op2 in Rd
MOV	Rd,Op2	Move (Copy) Op2 to Rd
ORR	Rd, Rn,Op2	OR Rn with Op2 and place the result in Rd
RSB	Rd, Rn,Op2	Subtract Rn from Op2 and place the result in Rd
RSC	Rd, Rn,Op2	Subtract Rn from Op2 with carry and place the result in Rd
SBC	Rd, Rn,Op2	Subtract Op2 from Rn with carry and place the result in Rd
SUB	Rd, Rn,Op2	Subtract Op2 from Rn and place the result in Rd
TEQ	Rn,Op2	Exclusive-OR Rn with Op2 and set the status bits of CPSR
TST	Rn,Op2	AND Rn with Op2 and set the status bits of CPSR
* Op2 can be an immediate 8-bit value #K which can be 0–255 in decimal, (00–FF in hex). Op2 can also be a register Rm. Rd, Rn and Rm are any of the general purpose registers		
** CPSR is discussed later in this chapter		

Unconditional Branch (JUMP) Instructions

- The unconditional branch is a jump in which control is transferred unconditionally to the target location.
- In the ARM there are two unconditional branches:
 - B (Branch)
 - BX (Branch and Exchange)

B (Branch)

- B (Branch) is an unconditional jump that can go to any memory location within the $\pm 32\text{M}$ byte address range.
- B has different usages like implementing if/else, while, and for instructions.
- B is also used to create “branch to itself” in order to keep the program from running away:

HERE B HERE

All branches are short branches (Jump)

- Note that all branch instructions (conditional and unconditional) are short jumps, meaning the address of the target must be within $\pm 32\text{M}$ bytes of the program counter (PC).
- That means short jumps cannot cover the entire address space of 4G bytes (0x00000000 to 0xFFFFFFFF).

BX (Branch and Exchange)

- To branch beyond the address range of $\pm 32\text{M}$ Bytes, we use BX instruction.

Syntax:

```
BX Rn ; Rn can be any register between R0- R14
```

- BX is commonly used to call for subroutines.

BX LR

- Return from Subroutine call

END

- Ends the program.
- Always need one at the end of the code

Example

```
1      EXPORT __main
2      AREA myfirstcode, CODE, READONLY
3      __main
4      MOV R1, #0x25    ; R1 = 0x25
5      MOV R2, #0x34    ; R2 = 0x34
6      ADD R3, R2, R1   ; R3 = R2 + R1
7      LDR R4, =0x20000000
8      STR R3, [R4]     ; Store R3 in [R4]
9      LDR R5, [R4]     ; load [R4] in R5
10     HERE    B      HERE    ; Stay here forever
11     END
```

Disassembly Window

Memory Address Machine Code

Disassembly			
		<u>main:</u>	
➡ 0x0000039C	F04F0125	MOV	r1, #0x25
0x000003A0	F04F0234	MOV	r2, #0x34
0x000003A4	EB020301	ADD	r3, r2, r1
0x000003A8	F04F5400	MOV	r4, #0x20000000
0x000003AC	6023	STR	r3, [r4, #0x00]
0x000003AE	6825	LDR	r5, [r4, #0x00]
0x000003B0	E7FE	B	0x000003B0

Memory 1

Address: 0x0000039C

0x0000039C:	4F	F0	25	01	4F	F0	34	02	02	EB	01	03	4F	F0
0x000003AA:	00	54	23	60	25	68	FE	E7	00	00	00	00	00	00
0x000003B8:	00	00	00	00	00	00	00	00	00	00	00	00	00	00
0x000003C6:	00	00	00	00	00	00	00	00	00	00	00	00	00	00
0x000003D4:	00	00	00	00	00	00	00	00	00	00	00	00	00	00
0x000003E2:	00	00	00	00	00	00	00	00	00	00	00	00	00	00
0x000003F0:	00	00	00	00	00	00	00	00	00	00	00	00	00	00
0x000003FE:	00	00	00	00	00	00	00	00	00	00	00	00	00	00
0x0000040C:	00	00	00	00	00	00	00	00	00	00	00	00	00	00

Call Stack + Locals | **Memory 1**

Memory Address Machine Code

Disassembly	
➡ 0x0000039C	F04F0125
0x000003A0	F04F0234
0x000003A4	EB020301
0x000003A8	F04F5400
0x000003AC	6023
0x000003AE	6825
0x000003B0	E7FE

Takeaways

- ARM Microcontroller Architecture
- RISC vs CISC
- Memory types
- Memory Map
- Basic Assembly